

COMPLETE JAVA PROGRAMMING NOTES

1. What is Java?

Java is a high-level, object-oriented programming language.

It is used to create:

- Desktop applications
- Web applications
- Backend systems
- Android applications
- Enterprise software

Java is popular because the same Java program can run on different operating systems.

Main Features of Java

1. **Simple** — Java is easier to learn than many low-level languages.
 2. **Object-Oriented** — Java programs are mainly built using classes and objects.
 3. **Platform Independent** — Java code can run on Windows, Linux, and macOS.
 4. **Secure** — Java provides many security features.
 5. **Robust** — Java handles errors and memory carefully.
 6. **Multithreaded** — Java can perform multiple tasks at the same time.
 7. **Portable** — Java programs can easily move from one system to another.
-

2. JDK, JRE, and JVM

JVM — Java Virtual Machine

JVM runs Java bytecode.

It converts bytecode into machine code that the computer can understand.

Simple flow:

Java Program → Bytecode → JVM → Machine Code

JRE — Java Runtime Environment

JRE provides everything needed to run a Java program.

It contains:

- JVM
- Java libraries

JRE is mainly used to run Java programs.

JDK — Java Development Kit

JDK provides everything needed to create and run Java programs.

It contains:

- JRE
- JVM
- Java compiler
- Development tools

Simple Difference

JVM = Runs bytecode

JRE = Provides environment to run Java programs

JDK = Provides tools to create and run Java programs

3. How a Java Program Works

Suppose we create:

```
Hello.java
```

First, the Java compiler compiles the program:

```
Hello.java → Hello.class
```

The `.class` file contains bytecode.

Then:

```
Bytecode → JVM → Machine Code
```

This is why Java is called:

Write Once, Run Anywhere.

4. Basic Java Program

```
class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello Java");  
  
    }  
}
```

Explanation

`class Main`

Creates a class named Main.

`public static void main(String[] args)`

This is the main method. The execution of the program starts from here.

`System.out.println()`

Prints something on the screen.

5. Variables in Java

A variable is a container used to store data.

Example:

```
int age = 20;
```

Here:

- `int` = data type
 - `age` = variable name
 - `20` = value
-

Types of Variables

1. Local Variable

A variable created inside a method is called a local variable.

```
class Main {  
    void show() {  
        int age = 20;  
        System.out.println(age);  
    }  
}
```

The variable `age` can only be used inside the `show()` method.

2. Instance Variable

A variable created inside a class but outside methods is called an instance variable.

```
class Student {  
    int age;  
}
```

Every object gets its own copy of an instance variable.

3. Static Variable

A variable declared using the `static` keyword is called a static variable.

```
class Student {  
    static String college = "DHSK College";  
}
```

A static variable is shared by all objects.

6. Data Types in Java

A data type tells Java what kind of data a variable can store.

Primitive Data Types

Java has 8 primitive data types.

byte

Stores small integer values.

```
byte age = 20;
```

short

Stores integer values larger than byte.

```
short number = 1000;
```

int

Most commonly used for whole numbers.

```
int marks = 90;
```

long

Stores very large whole numbers.

```
long population = 1000000000L;
```

float

Stores decimal numbers.

```
float price = 10.5f;
```

double

Stores larger and more accurate decimal numbers.

```
double salary = 50000.50;
```

char

Stores one character.

```
char grade = 'A';
```

boolean

Stores only two values:

```
true  
false
```

Example:

```
boolean isJavaEasy = true;
```

7. Methods in Java

A method is a block of code that performs a specific task.

Example:

```
class Main {  
  
    static void greet() {  
  
        System.out.println("Hello");  
  
    }  
  
    public static void main(String[] args) {  
  
        greet();  
  
    }  
}
```

The `greet()` method prints Hello.

Method with Parameters

```
class Main {  
  
    static void add(int a, int b) {  
  
        System.out.println(a + b);  
  
    }  
  
    public static void main(String[] args) {  
  
        add(10, 20);  
  
    }  
}
```

Output:

30

Method with Return Value

```
class Main {  
  
    static int add(int a, int b) {  
  
        return a + b;  
  
    }  
  
    public static void main(String[] args) {  
  
        int result = add(10, 20);  
  
        System.out.println(result);  
  
    }  
}
```

8. Class and Object

Class

A class is a blueprint or design for creating objects.

Example:

```
class Student {  
  
    String name;  
    int age;  
  
}
```

Object

An object is a real instance of a class.

Example:

```
class Student {  
  
    String name;  
    int age;  
  
}  
  
class Main {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
  
        s1.name = "Manish";  
        s1.age = 20;  
  
        System.out.println(s1.name);  
  
    }  
}
```

Here:


```
Student s1 = new Student();
```

creates an object.

9. Constructors

A constructor is a special method that is automatically called when an object is created.

A constructor:

- Has the same name as the class
- Has no return type
- Runs automatically when an object is created

Example:

```
class Student {  
  
    Student() {  
  
        System.out.println("Object created");  
  
    }  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
  
    }  
}
```

Output:

```
Object created
```

Parameterized Constructor

A constructor that receives values is called a parameterized constructor.

```
class Student {  
  
    Student(String name) {
```

```
        System.out.println(name);
    }

    public static void main(String[] args) {

        Student s1 = new Student("Manish");

    }
}
```

Constructor Overloading

When one class has multiple constructors with different parameters, it is called constructor overloading.

```
class Student {

    Student(int a, int b) {

        System.out.println(a + b);

    }

    Student(int a, int b, int c) {

        System.out.println(a + b + c);

    }

}
```

Java decides which constructor to call based on the arguments.

10. Method Overloading

Method overloading means creating multiple methods with:

- The same name
- Different parameters

Example:

```
class Calculator {

    void add(int a, int b) {
```

```
        System.out.println(a + b);
    }

    void add(int a, int b, int c) {

        System.out.println(a + b + c);

    }
}
```

This is called **compile-time polymorphism**.

11. Encapsulation

Encapsulation means hiding data and controlling how it is accessed.

We normally use:

- Private variables
- Getters
- Setters

Example:

```
class Student {

    private int age;

    public void setAge(int age) {

        this.age = age;

    }

    public int getAge() {

        return age;

    }

}
```

Using the class:

```
class Main {
```

```
public static void main(String[] args) {  
  
    Student s1 = new Student();  
  
    s1.setAge(20);  
  
    System.out.println(s1.getAge());  
  
}  
}
```

Why Use Encapsulation?

It:

- Protects data
- Gives better control over data
- Makes code safer

12. Inheritance

Inheritance means one class gets properties and methods from another class.

The `extends` keyword is used.

Example:

```
class Animal {  
  
    void eat() {  
  
        System.out.println("Animal is eating");  
  
    }  
}  
  
class Dog extends Animal {  
  
    void bark() {  
  
        System.out.println("Dog is barking");  
  
    }  
}
```

Now the Dog class can use both:

```
eat()  
bark()
```

Types of Inheritance

Single Inheritance

One child class inherits from one parent class.

```
Animal  
↓  
Dog
```

Multilevel Inheritance

A class inherits from a class that already inherited another class.

```
Animal  
↓  
Dog  
↓  
Puppy
```

Hierarchical Inheritance

Multiple child classes inherit from one parent class.

```
    Animal  
   /  
Dog  Cat
```

Java does not support multiple inheritance using classes.

13. Polymorphism

Polymorphism means:

One thing can have many forms.

Java has two main types.

1. Compile-Time Polymorphism

Achieved using method overloading.

```
void add(int a, int b)

void add(int a, int b, int c)
```

2. Runtime Polymorphism

Achieved using method overriding.

Example:

```
class Animal {

    void sound() {

        System.out.println("Animal makes sound");

    }

}

class Dog extends Animal {

    void sound() {

        System.out.println("Dog barks");

    }

}
```

14. Method Overriding

Method overriding happens when a child class creates its own version of a parent class method.

The method must have:

- Same name
- Same parameters
- Same or compatible return type

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

The Dog class overrides the `sound()` method.

15. Abstraction

Abstraction means:

Showing important things and hiding unnecessary details.

Example:

When you drive a car, you use the steering wheel and pedals.

You do not need to know exactly how the engine works internally.

Java provides abstraction using:

- Abstract classes
 - Interfaces
-

16. Abstract Class

A class declared using the `abstract` keyword is called an abstract class.

Example:

```
abstract class Animal {  
  
    abstract void sound();  
  
}
```

An abstract method has no body.

A child class must provide the body.

```
class Dog extends Animal {  
  
    void sound() {  
  
        System.out.println("Dog barks");  
  
    }  
}
```

We cannot directly create an object of an abstract class.

Wrong:

```
Animal a = new Animal();
```

17. Interface

An interface is a blueprint that tells a class what methods it must implement.

Example:

```
interface Animal {  
  
    void sound();  
  
}
```

A class uses the `implements` keyword.

```
class Dog implements Animal {  
  
    public void sound() {
```



```
        System.out.println("Dog barks");  
    }  
}
```

Abstract Class vs Interface

Abstract class uses:

```
extends
```

Interface uses:

```
implements
```

A class can implement multiple interfaces.

18. Access Modifiers

Access modifiers control where variables, methods, and classes can be accessed.

Java has four main access levels.

public

Can be accessed from anywhere.

```
public int age;
```

private

Can only be accessed inside the same class.

```
private int age;
```

protected

Can be accessed:

- Inside the same package
- By child classes

```
protected int age;
```

default

When no access modifier is written.

```
int age;
```

It can mainly be accessed inside the same package.

19. final Keyword

The `final` keyword is used to stop something from changing.

Final Variable

The value cannot be changed.

```
final int age = 20;
```

This is wrong:

```
age = 30;
```

Final Method

A final method cannot be overridden.

```
final void show() {  
  
}
```

Final Class

A final class cannot be inherited.

```
final class Animal {  
  
}
```

20. Packages in Java

A package is used to group related classes.

It works like a folder.

Example:

```
package mypack;
```

Why Use Packages?

Packages help:

- Organize code
- Avoid naming conflicts
- Control access

21. Exception Handling

An exception is an unexpected problem that happens while a program is running.

Examples:

- Dividing by zero
- Accessing an invalid array position
- Opening a file that does not exist

Exception handling prevents the program from suddenly stopping.

22. Types of Exceptions

Checked Exception

Checked exceptions are checked by the compiler.

Example:

```
IOException  
FileNotFoundException
```

Java forces us to handle them.

Unchecked Exception

Unchecked exceptions happen while the program is running.

Examples:

```
ArithmeticException  
NullPointerException  
ArrayIndexOutOfBoundsException
```

Error

Errors are serious problems.

Example:

```
OutOfMemoryError  
StackOverflowError
```

Normally, programs do not handle errors.

23. try and catch

The `try` block contains risky code.

The `catch` block handles the exception.

Example:

```
class Main {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
        }  
    }  
}
```

```
    }

    catch (ArithmeticException e) {

        System.out.println("Cannot divide by zero");

    }

}

}
```

Instead of crashing, the program prints:

```
Cannot divide by zero
```

24. Multiple catch Blocks

One `try` block can have multiple `catch` blocks.

Example:

```
try {

    int arr[] = new int[5];

    arr[10] = 50;

}

catch (ArithmeticException e) {

    System.out.println("Math error");

}

catch (ArrayIndexOutOfBoundsException e) {

    System.out.println("Array error");

}
```

Java runs the catch block that matches the exception.

25. Nested try Block

A try block inside another try block is called a nested try block.

Example:

```
try {  
    System.out.println("Outer try");  
  
    try {  
        int a = 10 / 0;  
    }  
  
    catch (ArithmeticException e) {  
        System.out.println("Inner exception handled");  
    }  
}  
  
catch (Exception e) {  
    System.out.println("Outer exception handled");  
}
```

26. finally Block

The `finally` block runs whether an exception happens or not.

Example:

```
try {  
    int a = 10 / 2;  
}  
  
catch (Exception e) {  
    System.out.println("Error");  
}
```

```
}  
  
finally {  
    System.out.println("Always runs");  
}
```

The `finally` block is commonly used for cleanup work.

27. throw Keyword

The `throw` keyword is used to manually create an exception.

Example:

```
int age = 15;  
  
if (age < 18) {  
    throw new ArithmeticException("Not eligible");  
}
```

Simple meaning:

throw = I want to create an exception here.

28. throws Keyword

The `throws` keyword tells Java:

This method may produce an exception. The caller should handle it.

Example:

```
import java.io.*;  
  
class Main {  
    static void readFile() throws IOException {  
        FileReader file = new FileReader("data.txt");  
    }  
}
```

```

    }

    public static void main(String[] args) {

        try {

            readFile();

        }

        catch (IOException e) {

            System.out.println("File problem");

        }

    }

}

```

throw vs throws

throw

- Used inside a method
- Actually creates an exception

throws

- Used in method declaration
- Warns that a method may produce an exception

29. Exception Propagation

Exception propagation means an exception moves from one method to another until it is handled.

Example:

```

class Main {

    static void method1() {

        int a = 10 / 0;

    }

    static void method2() {

```



```

        method1();

    }

    public static void main(String[] args) {

        try {

            method2();

        }

        catch (Exception e) {

            System.out.println("Exception handled");

        }

    }

}

```

Flow:

```

method1()
  ↓
method2()
  ↓
main()
  ↓
catch

```

30. Custom Exception

A custom exception is an exception created by the programmer.

Example:

```

class AgeException extends Exception {

    AgeException(String message) {

        super(message);

    }

}

```

Using it:

```
class Main {  
  
    public static void main(String[] args) {  
  
        int age = 15;  
  
        try {  
  
            if (age < 18) {  
  
                throw new AgeException("Age must be 18 or above");  
  
            }  
  
        }  
  
        catch (AgeException e) {  
  
            System.out.println(e.getMessage());  
  
        }  
  
    }  
  
}
```

31. Exception Handling with Method Overriding

When a child class overrides a parent method, there are rules about exceptions.

A child method:

- Can throw the same checked exception
- Can throw a smaller child exception
- Cannot throw a broader checked exception than the parent method

Example:

```
class Parent {  
  
    void show() throws Exception {  
  
    }  
  
}  
  
class Child extends Parent {
```

```
void show() throws Exception {  
  
    }  
}
```

32. Collection Framework

The Collection Framework is a group of classes and interfaces used to store and manage multiple objects.

Main parts:

```
Collection  
|  
|-- List  
|  
|-- Set  
|  
|-- Queue
```

Map is also part of the framework but does not directly extend Collection.

33. ArrayList

ArrayList stores multiple values in a dynamic array.

Example:

```
import java.util.ArrayList;  
  
class Main {  
  
    public static void main(String[] args) {  
  
        ArrayList<String> names = new ArrayList<>();  
  
        names.add("Manish");  
        names.add("Rahul");  
  
        System.out.println(names);  
    }  
}
```

```
}  
}
```

Features

- Allows duplicate values
 - Maintains insertion order
 - Fast for accessing elements
 - Size can grow automatically
-

34. LinkedList

LinkedList stores elements as connected nodes.

Example:

```
import java.util.LinkedList;  
  
class Main {  
  
    public static void main(String[] args) {  
  
        LinkedList<String> names = new LinkedList<>();  
  
        names.add("Manish");  
        names.add("Rahul");  
  
        System.out.println(names);  
  
    }  
}
```

Features

- Allows duplicates
 - Maintains insertion order
 - Good for frequent insertion and deletion
-

35. ListIterator

ListIterator is used to move through a list.

It can move:

- Forward

- Backward

Example:

```
import java.util.*;

class Main {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("A");
        names.add("B");

        ListIterator<String> it = names.listIterator();

        while (it.hasNext()) {

            System.out.println(it.next());

        }

    }
}
```

36. HashSet

HashSet stores unique values.

Example:

```
import java.util.HashSet;

class Main {

    public static void main(String[] args) {

        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(10);

        System.out.println(numbers);

    }
}
```

```
}  
}
```

The duplicate `10` is not stored twice.

Features

- No duplicate values
- Does not guarantee insertion order
- Fast operations

37. LinkedHashSet

LinkedHashSet stores unique values and maintains insertion order.

Example:

```
import java.util.LinkedHashSet;  
  
class Main {  
  
    public static void main(String[] args) {  
  
        LinkedHashSet<Integer> numbers = new LinkedHashSet<>();  
  
        numbers.add(30);  
        numbers.add(10);  
        numbers.add(20);  
  
        System.out.println(numbers);  
  
    }  
}
```

Output keeps the insertion order.

38. TreeSet

TreeSet stores unique values in sorted order.

Example:

```
import java.util.TreeSet;
```

```
class Main {  
  
    public static void main(String[] args) {  
  
        TreeSet<Integer> numbers = new TreeSet<>();  
  
        numbers.add(30);  
        numbers.add(10);  
        numbers.add(20);  
  
        System.out.println(numbers);  
  
    }  
}
```

Output:

```
[10, 20, 30]
```

39. PriorityQueue

PriorityQueue processes elements according to priority.

By default, smaller values get higher priority.

Example:

```
import java.util.PriorityQueue;  
  
class Main {  
  
    public static void main(String[] args) {  
  
        PriorityQueue<Integer> queue = new PriorityQueue<>();  
  
        queue.add(30);  
        queue.add(10);  
        queue.add(20);  
  
        System.out.println(queue.remove());  
  
    }  
}
```

Output:

40. ArrayDeque

ArrayDeque is a double-ended queue.

We can add and remove elements from both sides.

Example:

```
import java.util.ArrayDeque;

class Main {

    public static void main(String[] args) {

        ArrayDeque<Integer> deque = new ArrayDeque<>();

        deque.addFirst(10);
        deque.addLast(20);

        System.out.println(deque);

    }
}
```

41. Map Interface

Map stores data in key-value pairs.

Example:

```
101 → Manish
102 → Rahul
```

Each key should be unique.

42. HashMap

HashMap stores data as key-value pairs.

Example:

```
import java.util.HashMap;

class Main {

    public static void main(String[] args) {

        HashMap<Integer, String> students = new HashMap<>();

        students.put(101, "Manish");
        students.put(102, "Rahul");

        System.out.println(students);

    }
}
```

Accessing a value:

```
System.out.println(students.get(101));
```

Output:

```
Manish
```

Features

- Stores key-value pairs
- Keys are unique
- Values can be duplicated
- Does not guarantee insertion order

43. Comparable

Comparable is used to define the natural sorting order of objects.

It uses:

```
compareTo()
```

Example:

```
class Student implements Comparable<Student> {  
  
    int age;  
  
    public int compareTo(Student s) {  
  
        return this.age - s.age;  
  
    }  
}
```

44. Comparator

Comparator is used to create custom sorting rules.

It uses:

```
compare()
```

Simple difference:

Comparable

- Natural sorting
- Uses `compareTo()`
- Sorting logic is usually inside the class

Comparator

- Custom sorting
 - Uses `compare()`
 - Can create multiple sorting rules
-

45. Multithreading

A thread is a small unit of work inside a program.

Multithreading means performing multiple tasks at the same time.

Example:

```
Task 1 → Download file
```

Task 2 → Play music

Task 3 → Show animation

All these tasks can work together.

46. Creating a Thread

Using the Thread Class

```
class MyThread extends Thread {  
  
    public void run() {  
  
        System.out.println("Thread is running");  
  
    }  
}  
  
class Main {  
  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread();  
  
        t1.start();  
  
    }  
}
```

The `run()` method contains the work.

The `start()` method starts a new thread.

47. Thread Synchronization

Thread synchronization means allowing only one thread at a time to access an important shared resource.

Example:

```
synchronized void print() {  
  
    System.out.println("Safe method");  
}
```

```
}
```

Why Is It Needed?

Suppose two threads try to change the same bank balance at the same time.

This may create incorrect results.

Synchronization prevents this problem.

48. Sleeping a Thread

The `sleep()` method pauses a thread for some time.

Example:

```
try {  
    Thread.sleep(2000);  
}  
  
catch (InterruptedException e) {  
    System.out.println(e);  
}
```

`2000` means 2000 milliseconds.

So the thread sleeps for 2 seconds.

49. wait() Method

The `wait()` method makes a thread wait until another thread tells it to continue.

Commonly used methods:

```
wait()  
notify()  
notifyAll()
```

Simple meaning:

`wait()` → Stop and wait

`notify()` → Wake up one waiting thread

`notifyAll()` → Wake up all waiting threads

50. QUICK REVISION OF JAVA

Java Basics

Variable → Stores data

Data Type → Defines the type of data

Method → Performs a task

Class → Blueprint

Object → Real instance of a class

Constructor → Runs when an object is created

OOP Concepts

Encapsulation → Hiding and protecting data

Inheritance → Getting features from another class

Polymorphism → One thing, many forms

Abstraction → Hiding unnecessary details

Exception Handling

`try` → Risky code

`catch` → Handles exception

`finally` → Always runs

`throw` → Manually creates an exception

throws → Declares a possible exception

Collections

ArrayList → Dynamic array

LinkedList → Connected nodes

HashSet → Unique unordered values

LinkedHashSet → Unique values in insertion order

TreeSet → Unique sorted values

PriorityQueue → Elements processed by priority

ArrayDeque → Add/remove from both ends

HashMap → Key-value pairs

Multithreading

Thread → Small unit of work

start() → Starts a thread

run() → Contains thread work

sleep() → Pauses a thread

synchronized → Controls shared access

wait() → Makes a thread wait

notify() → Wakes one waiting thread

FINAL LEARNING MAP

Java Basics

↓

JDK, JRE, JVM

↓

Variables and Data Types

↓

Methods
↓
Classes and Objects
↓
Constructors
↓
Method Overloading
↓
Encapsulation
↓
Inheritance
↓
Polymorphism
↓
Method Overriding
↓
Abstraction
↓
Abstract Classes
↓
Interfaces
↓
Access Modifiers
↓
final Keyword
↓
Packages
↓
Exception Handling
↓
Collection Framework
↓
Multithreading